

API Security Risk Analysis

JSONPlaceholder and ReqRes

Read only assessment for Future Interns Cyber Security Task 3 2026

Prepared by R Aniruthvishwak

9 September 2026

Purpose

This report examines public API requests made in Postman, explains what the observed behavior means for security, and recommends practical improvements. It covers authentication, access control, response data, transport protection, rate-limit indicators and simple invalid-input handling.

Main conclusion

The tested JSONPlaceholder endpoint returned data over HTTP. This creates a Low-risk transport weakness in this public test-data context. The protected ReqRes request rejected a missing key. No High- or Medium-risk vulnerability was confirmed in the limited assessment; untested controls must not be treated as proven secure.

Table of Contents

S.No.	Section	Page
1	Executive Summary	3
2	Scope Objectives and Method	4
3	Endpoint Test Results	5–6
4	Coverage of the Task 3 Risk Areas	6–7
5	Severity Method and OWASP Alignment	7
6	F01 – API Data Served Over HTTP	8–9
7	F02 – Header Information and Hardening	9–10
8	Authentication Data and Authorization	10–11
9	Rate Limits and Invalid Input	11–12
10	Remediation Limitations and Conclusion	13–14
11	References and Submission Requirements	14–15
12	Evidence Register	16–17
13	E01 – Postman Client Readiness	18
14	E02 – Project Workspace Organization	19
15	E03 – Baseline GET Response	20
16	E04 – No Auth Configuration	21
17	E05 – Response Headers (Top)	22
18	E06 – Response Headers (Bottom)	23
19	E07 – Single User No Auth Response	24
20	E08 – All Users No Auth Response	25
21	E09 – Second Object Access	26
22	E10 – Nonexistent User ID	27
23	E11 – Invalid User ID Format	28
24	E12 – Redirects Disabled	29
25	E13 – HTTP 200 Response Body	30
26	E14 – HTTP Response Headers (Top)	31
27	E15 – HTTP Response Headers (Bottom)	32
28	E16 – ReqRes No Auth Response	33
29	E17 – ReqRes Public Endpoint Body	34
30	E18 – ReqRes Public Endpoint Headers	35
31	E19 – ReqRes Missing API Key	36–38

Audience: Future Interns reviewers and technical or business readers evaluating a beginner API security assessment.

Executive summary

I examined selected endpoints on JSONPlaceholder and ReqRes using ordinary GET requests. The review was intended to identify observable security risks without attempting exploitation, crossing account boundaries or modifying data. Nineteen screenshots preserve the request settings and the responses observed during the project.

The clearest finding is the delivery of the JSONPlaceholder /users/1 response over unencrypted HTTP while automatic redirects were disabled. The service documents support for both HTTP and HTTPS; the observed behavior is therefore a deliberate compatibility choice, but it still lacks confidentiality and integrity on the HTTP path. [S1]

ID	Result	Assessment
F-01	API data available over HTTP	Low risk Confirmed on tested endpoint
F-02	Technology names exposed in headers	Low risk Hardening opportunity
O-01	Broad public user-like response fields	Informational Expected test-data behavior
C-01	Protected ReqRes path rejects missing key	Positive control 401 observed
C-02	Unknown and nonnumeric IDs return 404	Positive observation Two inputs only
O-02	Rate-limit headers present	Enforcement not assessed

Business meaning

No real customer-data breach, account takeover or service disruption was demonstrated. In a production SaaS API, returning private records without access checks could expose customers, and sending tokens over HTTP could enable account misuse. These are conditional business scenarios, not events established by this assessment.

Priorities

- Use HTTPS directly in clients and stop returning API data over HTTP in a hardened deployment.
- Keep public demonstration records separate from private user data, and enforce appropriate access rules when adapting the design to production.
- Retain the observed missing-key rejection and validate authorization and resource controls in a separately approved test environment.

Overall observed risk: Low within the tested public-data scope. This is not an overall security rating for either provider or a certification that untested services are secure.

Scope objectives and method

Project objective

The objective was to select test APIs, review their documented purpose, inspect responses in Postman, identify supported risks, explain business impact and recommend remediation. Postman acted as the API client: it sent the request that an application would normally send and displayed the server's response.

Targets and documentation

JSONPlaceholder was the primary target. Its documentation identifies it as a fake API for testing and lists public resource routes. ReqRes was used for comparison: its public user response was contrasted with a path that demanded an API key. Documentation and response evidence were considered together; behavior of one endpoint was not generalized to the whole service. [S1-S3]

Environment and tools actually used

- Kali Linux virtual machine on VMware Fusion; Postman Desktop in the signed-out Lightweight API Client workflow.
- Terminal commands to create directories and list evidence files.
- Word and PDF for reporting. Insomnia, Browser DevTools, Nmap, Burp Suite and ZAP were not used for these Task 3 endpoint checks.

Allowed work and boundaries

Only the selected testing-service URLs were reviewed. Requests used GET and no valid API credentials were supplied. No POST, PUT, PATCH or DELETE request, credential guessing, payload exploitation, authentication bypass, automated enumeration, flooding or denial-of-service test was performed. A publicly reachable API directory is not blanket permission to test every listed service.

The ReqRes check stopped at the missing-key error. The visible X-Reqres-Env: prod selector did not authorize access to a private collection or tenant. No private collection data was obtained.

Sequence and evidence method

The sequence was baseline request, No Auth configuration, header review, user and collection responses, object-ID comparison, two invalid-ID checks, HTTP behavior with redirects disabled, ReqRes public response, and a missing-key check. Screenshots were then inventoried and linked to the analysis using evidence IDs E01-E19.

Read-only is not the same as passive network observation. Sending a GET request actively contacts the service, even though the intended operation only reads data. The project used low-volume read-only requests, not a passive packet capture.

Endpoint test results

Every row below is a GET request. The reported settings were No Auth with no intentionally supplied credentials. E12 records the redirect setting used specifically for the HTTP check. Times and response sizes are diagnostic observations, not security benchmarks.

Test	URL	Observed result	Evidence
T01	https://jsonplaceholder.typicode.com/posts/1	200 OK; one post	E03-E06
T02	https://jsonplaceholder.typicode.com/users/1	200 OK; one user-like record	E07
T03	https://jsonplaceholder.typicode.com/users	200 OK; array of records	E08
T04	https://jsonplaceholder.typicode.com/users/2	200 OK; different object, id 2	E09
T05	https://jsonplaceholder.typicode.com/users/999999	404 Not Found; {}	E10
T06	https://jsonplaceholder.typicode.com/users/abc	404 Not Found; {}	E11
T07	http://jsonplaceholder.typicode.com/users/1	200 OK; JSON body; redirects OFF	E12-E15
T08	https://reqres.in/api/users/2	200 OK; read-only demo record	E16-E18

Test	URL	Observed result	Evidence
T09	https://reqres.in/api/collections/products/records	401 Unauthorized; missing_api_key	E19

Reading the response

The status code states the HTTP outcome. The response body contains returned data or an error; headers describe aspects such as its content type, caching and browser behavior. A URL ending in /users/1 identifies a particular resource. /users identifies the collection.

Request configuration limitation

No Auth shows that Postman's authentication helper is disabled. It does not, by itself, prove that all outgoing credential headers and cookies are absent. The screenshots do not show every hidden request header. The conclusion uses the recorded test procedure and the service's own response, including its explicit missing_api_key error, rather than treating the client menu as a full wire capture.

Reproduction boundary

These rows describe the completed tests, not a request to repeat them against unrelated targets. Any future verification should remain limited to approved test services and should stop before bypass attempts or data-changing operations.

Coverage of the Task 3 risk areas

All six required risk areas are addressed below. Coverage means the topic was considered; it does not mean a vulnerability was found or that comprehensive testing was possible.

Required area	Work performed	Conclusion and remaining limit
Open endpoints	Public posts, user and collection requests; ReqRes demo comparison.	Observed public access. Expected for the selected demo routes; not automatically broken authentication.
Excessive data exposure	Reviewed field names and the collection response.	Broad user-like data observed. No unauthorized sensitive-field disclosure confirmed; O-01 is informational.

Required area	Work performed	Conclusion and remaining limit
Weak or missing tokens	Recorded No Auth and the protected missing-key response.	Missing key rejected. Token strength, expiry, rotation, signatures and revocation not tested.
Authorization and other users' data	Compared public IDs 1 and 2.	Different objects returned. No account or tenant boundary existed in the test; BOLA not confirmed.
Missing rate limiting	Inspected limit, remaining and reset headers.	Headers present, including cache indicators. No exhaustion test; enforcement remains unverified.
Input validation	Requested /users/999999 and /users/abc.	Both returned 404 and {}. No visible internal error; this does not prove schema validation or injection resistance.

Severity method

Severity reflects plausible likelihood and impact for the actual data and operation tested. High means a credible path to major compromise or disruption; Medium means material impact with narrower reach or additional conditions; Low means limited impact or a hardening gap. Informational and positive-control observations are not vulnerabilities and are not forced into Low ratings.

No CVSS score is assigned because this limited configuration review does not establish a complete exploit scenario. Hypothetical production impacts do not change the demonstrated target severity.

OWASP alignment

Authorization relates to API1:2023, authentication to API2:2023, field exposure to API3:2023, resource limits to API4:2023, and configuration to API8:2023. These are analysis categories, not declarations that each OWASP risk exists here. The 2023 property-authorization category incorporates the earlier excessive-data-exposure topic. [S4]

F01 API data served over HTTP

Severity: Low | Status: Confirmed configuration behavior | Evidence: E12-E15

GET <http://jsonplaceholder.typicode.com/users/1>
 Automatically follow redirects: OFF
 Observed response: 200 OK with the user JSON object

Observation and interpretation

The selected HTTP URL returned the API body directly. The settings screenshot confirms that Postman was not automatically following redirects, while the body and paired header screenshots show a 200 result. This supports the narrow conclusion that this endpoint was accessible through unencrypted HTTP on the observed request path.

JSONPlaceholder explicitly documents support for HTTP and HTTPS. This finding records the security tradeoff of that design rather than alleging that the service violated its published behavior. The records are public test data, which limits confidentiality impact. [S1]

Business impact and rating rationale

A party positioned on an untrusted network path could read or alter an HTTP exchange. For this test, the practical concern is chiefly response integrity: altered data could mislead a learning application. No interception or alteration was attempted, and no token or password was sent.

In a production workflow carrying credentials or private records, equivalent cleartext delivery could be High risk because it could expose accounts or confidential information. That scenario was not present in the test and is not counted as a High finding.

Recommended remediation

- Configure hardened API deployments to accept resource access over HTTPS only. Reject HTTP API calls, or use a body-free HTTPS redirect for deliberately supported public compatibility routes.
 - Configure clients with HTTPS base URLs from the start. A redirect cannot undo exposure of a credential already sent in the initial HTTP request.
 - Retain certificate validation and consider HSTS for browser clients once HTTPS deployment is complete. This is a deployment recommendation, not evidence that a full TLS audit was performed.
- [S5]

Retest acceptance criteria

With redirect following off, the HTTP resource must no longer return the record. The HTTPS URL should continue returning the intended public response with certificate validation enabled. Verification must not send real credentials over HTTP. Test other routes only within an explicitly approved scope.

F02 Header information and hardening

Severity: Low | Status: Confirmed disclosure | Evidence: E05-E06 and E14-E15

The observed JSONPlaceholder responses disclose technology names through X-Powered-By: Express, Server: cloudflare and Via values identifying a Heroku router. These assist basic reconnaissance but do not identify a vulnerable component version or prove that the origin server is exploitable.

Business impact and remediation

The impact is limited: an observer can form hypotheses about the serving stack. Remove optional identifying headers such as X-Powered-By where practical and review error output for unnecessary implementation details. Do not remove protocol-required intermediary metadata blindly. Patch and dependency management remain more important than concealing a framework name.

Retest by inspecting normal responses and errors for optional stack disclosures. Managed edge or Via identifiers may remain by design; document that decision. ReqRes's promotional X-Powered-By value is not evidence of an Express version.

Other header observations

Header or group	What the evidence supports
Content-Type and nosniff	JSON content type on JSONPlaceholder; nosniff visible on both services. Positive header observations, not proof of safe data handling.
Cache-Control and cache status	Public responses were cacheable and cache hits were visible. No private cache leak was demonstrated.
Access-Control-Allow-Credentials	true appears on JSONPlaceholder. Alone it does not establish a permissive origin policy or a CORS exploit.

Header or group	What the evidence supports
Rate-limit fields	Present on both services. Detailed enforcement limitations are discussed on page 9.
X-Frame-Options	DENY appears in the ReqRes view. This is browser-oriented and is not an authorization control.
HSTS and CSP	Not observed in the paired JSONPlaceholder baseline header views. Their absence is not promoted to a separate Medium finding for this JSON endpoint.

The ReqRes screenshot shows only part of its response-header list, so absent-header claims are not made for that service. Headers that primarily govern HTML rendering must be assessed in the context of how the API is consumed. [S5]

Authentication data and authorization

O01 Public user fields and data minimization

Status: Informational, expected public-data behavior. E07-E09 show name, username, email, address, geographic coordinates, phone, website and company fields. The collection view visibly contains multiple objects, although the screenshot is not a complete capture of every array element.

The response-field review identifies what would need a privacy decision in a real SaaS product. It does not establish excessive disclosure in this demo because no protected field or prohibited recipient was identified. A production owner should define response schemas by purpose and expose only fields the caller is permitted to receive.

Open access is not automatically broken authentication

The baseline No Auth setting and successful responses establish the intended public workflow. ReqRes also returned metadata describing `/api/users/2` as a read-only demo. Its public documentation supports anonymous access for that demo. This should not be described as bypassing login. [S3]

C01 Missing API key rejected

GET <https://reqres.in/api/collections/products/records>

X-Reqres-Env: prod

401 Unauthorized

error: missing_api_key

E19 records rejection when no key was intentionally supplied. The response names x-api-key as a requirement and returns help text rather than collection records. This is evidence of one enforced authentication check, not proof that the named collection exists, that every endpoint is protected, or that an authenticated caller would receive appropriate authorization.

Token entropy, JWT algorithms, claims, expiration, revocation, key rotation and secure storage were not examined because no issued token or key was obtained. A help message mentioning a bearer token is not a token-strength test.

Object access and BOLA

E07 and E09 show that IDs 1 and 2 select different objects. Broken Object Level Authorization requires an access boundary to be violated, not merely an identifier to be predictable. An authorized production assessment would need test accounts with known ownership or tenant rules; that comparison was outside this engagement. [S6]

Business implication: unauthorized object or field access in production could expose customer records. Recommended design controls are server-side checks on each protected resource and response field. These are preventive recommendations, not confirmed flaws in the two public records tested.

Rate limits and invalid input

02 Rate limit metadata observed

Response	Limit	Remaining	Reset	Cache indicator
JSONPlaceholder HTTPS /posts/1	1000	999	1783558083	HIT; Age 13453
JSONPlaceholder HTTP /users/1	1000	999	1788713985	HIT; Age 5202
ReqRes /api/users/2	40	38	1788998400	REVALIDATED

E06, E15 and E18 preserve these values. They show that the responses carry quota-related metadata, but the time window, identity scope and effective enforcement were not measured. A value of 40 is not evidence of 40 requests per second or per minute.

Caching is a material limitation. The HTTPS JSONPlaceholder reset value, if interpreted as Unix seconds, is 9 July 2026 at 00:48:03 UTC, earlier than the September response date shown. Cached or stale metadata is plausible, but its exact cause was not verified. Remaining: 999 must not be assumed to be the current client's live balance.

No request loop or quota exhaustion was performed, and no 429 result was captured. It would be unsupported to report either missing rate limiting or proven enforcement. In production, resource limits should be designed around request cost, caller identity and service capacity, then validated in an approved environment. [S7]

C02 Controlled responses to two invalid identifiers

Input	Status	Body	Interpretation
/users/9999999	404	{}	No matching resource returned
/users/abc	404	{}	Nonnumeric path returned no object

E10-E11 show no stack traces or internal paths. These are positive error-handling observations for two simple cases. They do not establish that the server validates integer types, applies all schema restrictions, or resists injection. A route can produce 404 through ordinary lookup or matching.

Retest guidance: use agreed invalid values and verify predictable error responses without private data. Review validation code or schemas separately. Keep abuse, load and injection tests outside this project's read-only scope.

Remediation limitations and conclusion

Recommended work order

The priorities below are recommendations for a hardened deployment, not commitments made by the API providers.

Priority	Action and proposed owner	Acceptance evidence
First	Platform engineer: make API resource access HTTPS-only.	HTTP no longer returns the record; HTTPS remains functional.
Next	API developer: remove optional framework disclosures.	Response review shows optional identifying header removed or a documented exception.
Before private data is introduced	Product owner and API developer: define public fields and enforce ownership rules.	Approved response schema and permitted/denied test-account matrix.
Separate approved review	AppSec and platform teams: validate token lifecycle and resource controls.	Scoped authentication tests, quota configuration review and controlled validation without public-service flooding.

Limitations

- Only selected GET paths were assessed. No valid credentials, user sessions or tenant-specific records were used.
- No source code, origin logs, dependency inventory, full TLS audit or backend configuration was reviewed.
- Screenshots are client-side observations and may show only part of a response. No HAR export, packet capture or complete outgoing-header record was collected.
- Caching limits quota interpretation. Absence of a tested failure is not proof that a control exists.
- The protected request stopped at 401. No login, valid-key, role or ownership test was performed.

- Findings describe this evidence set and assessment period; they are not a guarantee of later behavior.

Conclusion

The project demonstrates a read-only API risk analysis with traceable evidence. Two Low-risk findings are recorded: direct HTTP data delivery and technology-name disclosure. Public user fields are treated as a contextual observation rather than a privacy breach. ReqRes's missing-key rejection and the two controlled 404 responses are recorded as positive observations.

The practical lesson is to distinguish what a response proves from what remains unknown. Enforce secure transport, define intentional public access, and assess authorization and resource controls with the necessary approved accounts and environment before drawing broader conclusions.

References and submission requirements

Documentation was reviewed to interpret the services and select relevant security categories. The observed results remain grounded in E01-E19. The Future Interns task brief supplied with the project defines the submission requirements.

S1 JSONPlaceholder overview and supported routes

S2 JSONPlaceholder guide

S3 ReqRes public demo description

S4 OWASP API Security Top 10 2023

S5 OWASP REST Security Cheat Sheet

S6 OWASP API1 Broken Object Level Authorization

S7 OWASP API4 Unrestricted Resource Consumption

Deliverable alignment

Requirement	Location or submission action
API tested and methodology	Pages 3-4; nine documented GET test cases.
Risks, severity and business impact	Pages 5-9; actual and conditional production impact distinguished.

Requirement	Location or submission action
Remediation suggestions	Pages 6-10; acceptance criteria included.
Postman screenshots	Evidence register and all 19 figures on pages 12-31.
Public GitHub repository	Publish the report under report/ and originals under evidence/postman/ after review.
README	Describe the actual tools, scope, methodology, results and limits. Repository publication is a separate step; it is not demonstrated by this report.

Publication and portfolio note

Keep API keys, bearer tokens and session cookies out of public files. The visible user-like records in this evidence belong to the test datasets. Review screenshots and document metadata before publishing; API error-message placeholders are not usable credentials.

This project is an API security assessment, not a dashboard design for an agency. A LinkedIn summary should describe the APIs, safe method and evidence-based conclusions, and may tag Future Interns. No fictional client engagement or implemented remediation should be claimed.

Evidence register

Nineteen screenshots form the evidence set. The upload suffix (1) on the two HTTPS header files is retained in the source mapping below; their original project names omit that suffix. Every item appears as a complete figure on its own evidence page.

ID	Original project filename	Page
E01	00_postman_ready.png	13
E02	00_project_structure.png	14
E03	01_baseline_get_body.png	15
E04	02_no_auth_configuration.png	16
E05	03a_response_headers_top.png	17
E06	03b_response_headers_bottom.png	18
E07	04_single_user_no_auth_response.png	19
E08	05_all_users_no_auth_response.png	20
E09	06_second_object_access.png	21
E10	07_nonexistent_user_id.png	22
E11	08_invalid_user_id_format.png	23
E12	09a_redirects_disabled.png	24
E13	09b_http_200_response_body.png	25
E14	09c_http_response_headers_top.png	26
E15	09d_http_response_headers_bottom.png	27
E16	10_reqres_no_auth_response.png	28
E17	10a_reqres_public_endpoint_body.png	29
E18	10b_reqres_public_endpoint_headers.png	30

ID	Original project filename	Page
E19	11_reqres_missing_api_key.png	31

Source mapping: E05 uses 03a_response_headers_top(1).png; E06 uses 03b_response_headers_bottom(1).png. All other uploaded filenames match the project filenames above.

```
find ~/FutureInterns/Task3/evidence/postman \
-maxdepth 1 -type f -printf '%fn' | sort
```

This inventory command lists files directly inside the evidence folder and sorts their basenames. It does not validate image contents or prove that a test was performed. The screenshots and their recorded context provide that evidence.

E01 Postman client readiness

00_postman_ready.png

Establish the tool used to send requests and inspect responses.

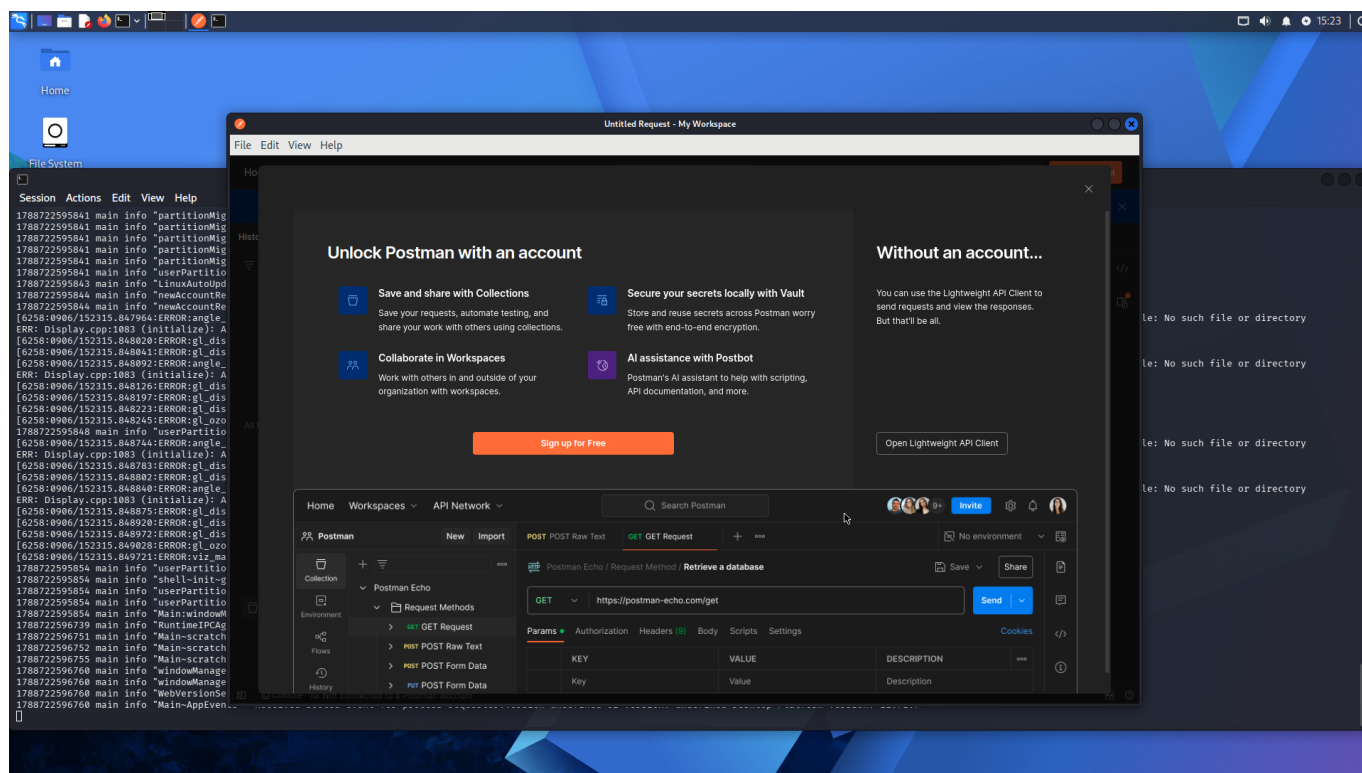


Figure 1. Postman client readiness.

Recorded result

Postman is open and offers the Lightweight API Client without an account. The visible onboarding panel explains that requests can be sent and responses viewed.

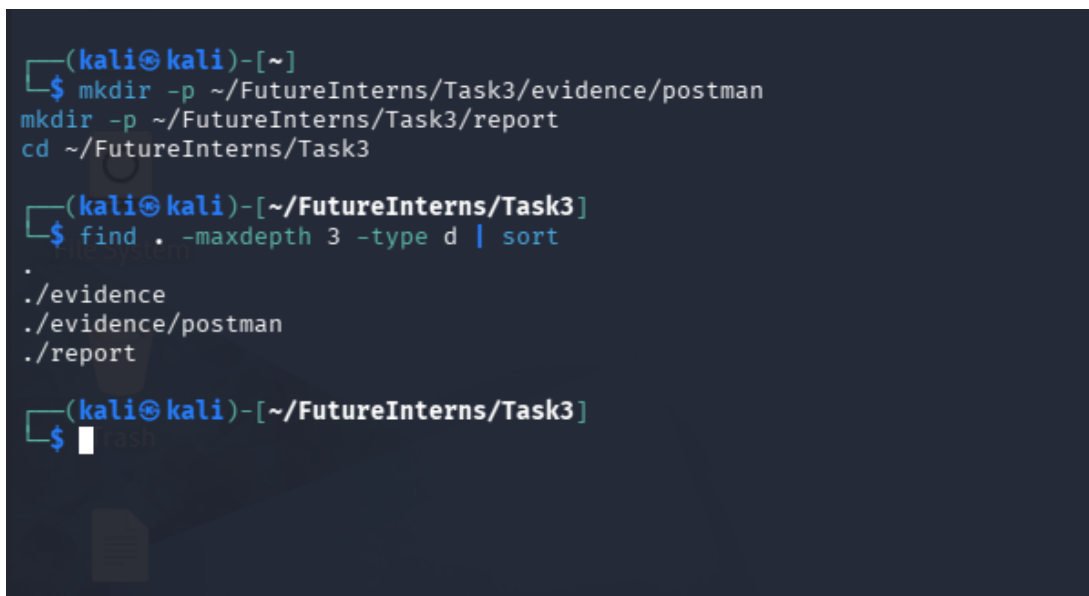
Security interpretation

This is setup evidence, not an API security result. Being signed out of Postman is different from using No Auth for a request: the first concerns the tool account, and the second concerns credentials sent to the target API.

E02 Project workspace organization

00_project_structure.png

Separate raw evidence from the report so results can be traced and reviewed.

A terminal window on a Kali Linux system. The prompt is (kali㉿kali)-[~]. The user enters \$ mkdir -p ~/FutureInterns/Task3/evidence/postman. The prompt changes to (kali㉿kali)-[~/FutureInterns/Task3]. The user enters \$ find . -maxdepth 3 -type d | sort. The output is: ./evidence, ./evidence/postman, and ./report. The prompt returns to (kali㉿kali)-[~/FutureInterns/Task3].

```
(kali㉿kali)-[~]  
$ mkdir -p ~/FutureInterns/Task3/evidence/postman  
mkdir -p ~/FutureInterns/Task3/report  
cd ~/FutureInterns/Task3  
  
(kali㉿kali)-[~/FutureInterns/Task3]  
$ find . -maxdepth 3 -type d | sort  
.  
./evidence  
./evidence/postman  
./report  
  
(kali㉿kali)-[~/FutureInterns/Task3]  
$
```

Figure 2. Project workspace organization.

Recorded result

The terminal shows mkdir -p for evidence/postman and report, followed by cd and a directory listing. The output contains ./evidence, ./evidence/postman and ./report.

Security interpretation

mkdir -p creates missing parent directories and tolerates existing directories. cd changes the working directory. find . -maxdepth 3 -type d lists directories up to three levels below the current location; sort places the results in order.

Commands recorded in the screenshot

```
mkdir -p ~/FutureInterns/Task3/evidence/postman
```

```
mkdir -p ~/FutureInterns/Task3/report
```

```
cd ~/FutureInterns/Task3
```

```
find . -maxdepth 3 -type d | sort
```


E03 Baseline response from the posts endpoint

01_baseline_get_body.png

Check connectivity and establish a normal successful API response before comparing other behavior.

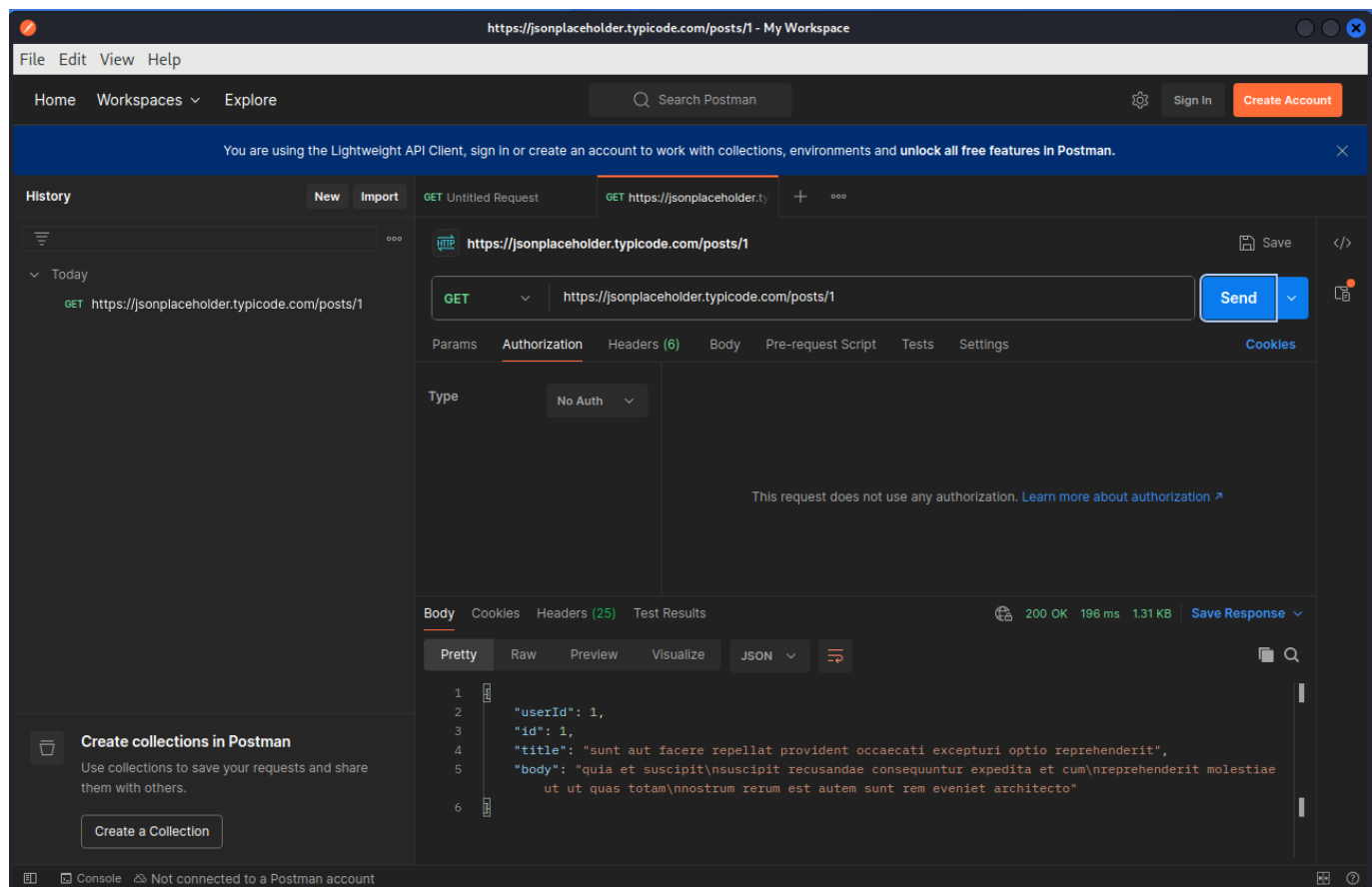


Figure 3. Baseline response from the posts endpoint.

Recorded result

GET `https://jsonplaceholder.typicode.com/posts/1` returned 200 OK. The JSON object contains `userId`, `id`, `title` and `body`. The Authorization pane displays No Auth.

Security interpretation

The client requested one public post and the service returned structured data. A 200 status means this request succeeded; it does not mean the API has passed a security audit. This request also supplies a baseline for later response-header inspection.

E04 Request authentication configuration

02_no_auth_configuration.png

Record the selected authentication option for the baseline request.

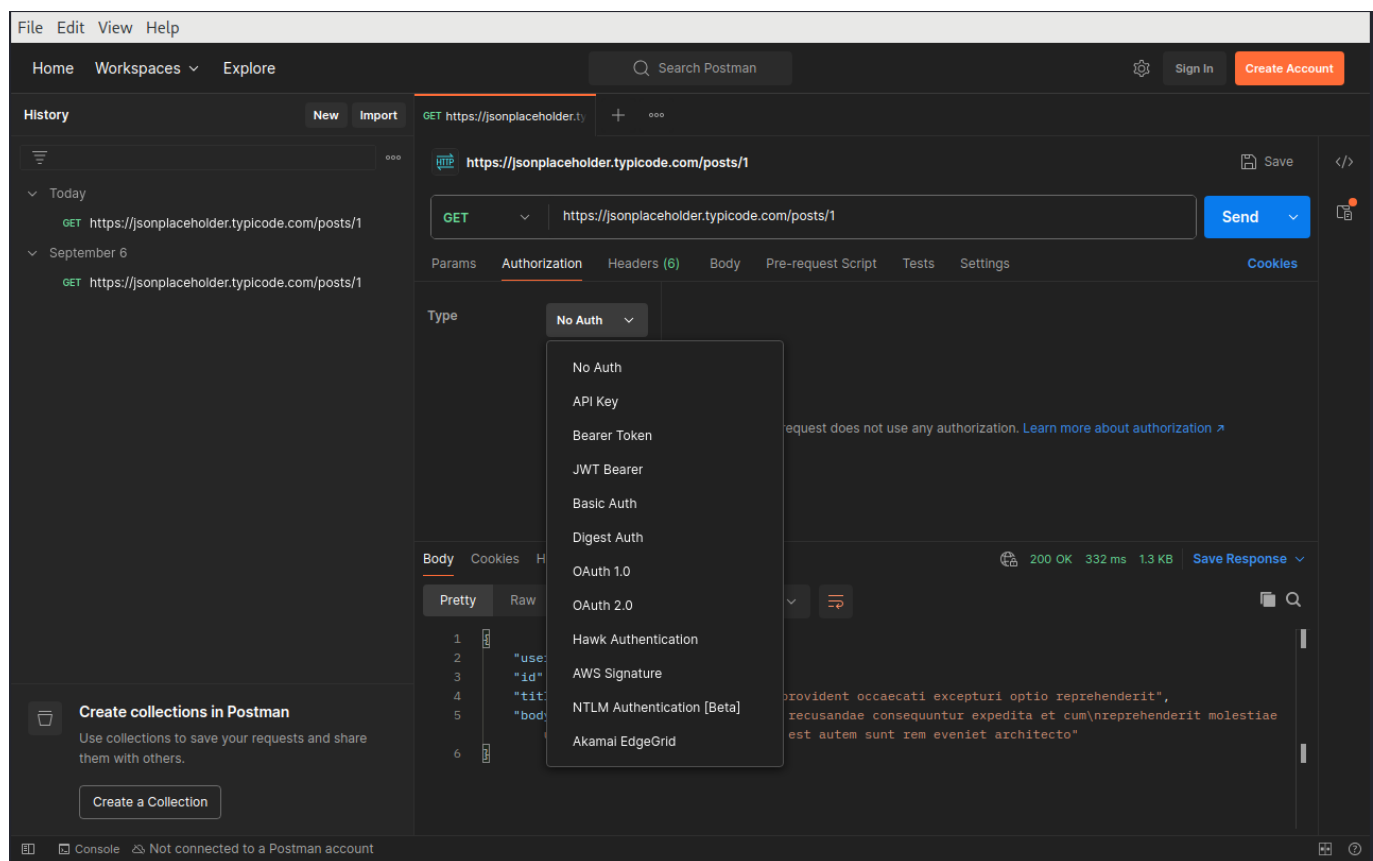


Figure 4. Request authentication configuration.

Recorded result

The Authorization type is No Auth. Other possible authentication types are visible in the menu, including API Key and Bearer Token.

Security interpretation

No Auth disables Postman's authentication helper for this request. By itself, that setting does not rule out manually added credential headers or cookies. No credentials were intentionally supplied during this project; a complete outgoing request export would provide stronger independent confirmation.

E05 HTTPS response headers in the upper view

03a_response_headers_top.png

Inspect response metadata for the baseline HTTPS request.

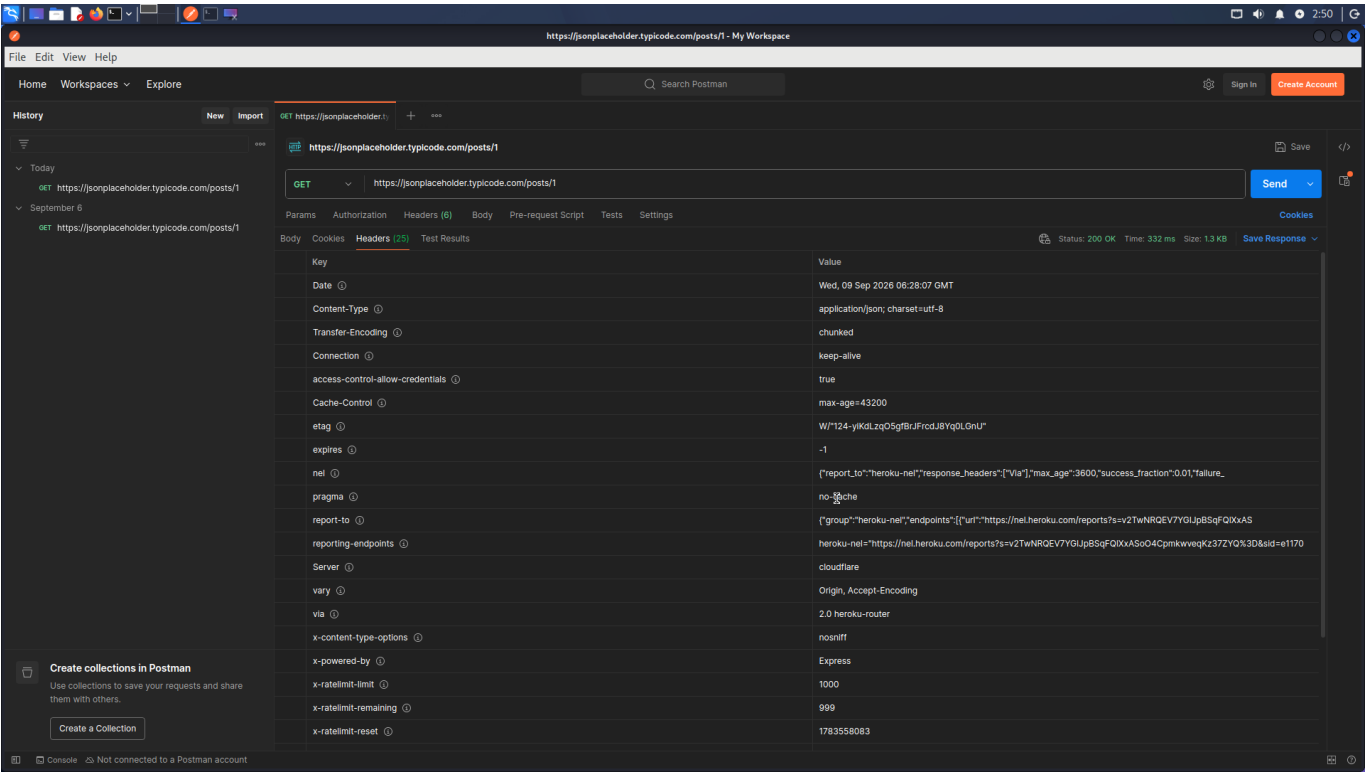


Figure 5. HTTPS response headers in the upper view.

Recorded result

Content-Type is application/json; charset=utf-8. Visible headers include Access-Control-Allow-Credentials: true, Cache-Control: max-age=43200, Server: cloudflare, Via: 2.0 heroku-router, X-Content-Type-Options: nosniff and X-Powered-By: Express.

Security interpretation

The content type and nosniff setting are useful browser-facing controls. Infrastructure names support F-02. Access-Control-Allow-Credentials alone does not prove a CORS vulnerability. No browser-origin variation or credentialed cross-origin test was performed.

E07 Single user record returned without configured authentication

04_single_user_no_auth_response.png

Review the kinds of fields exposed by a user resource.

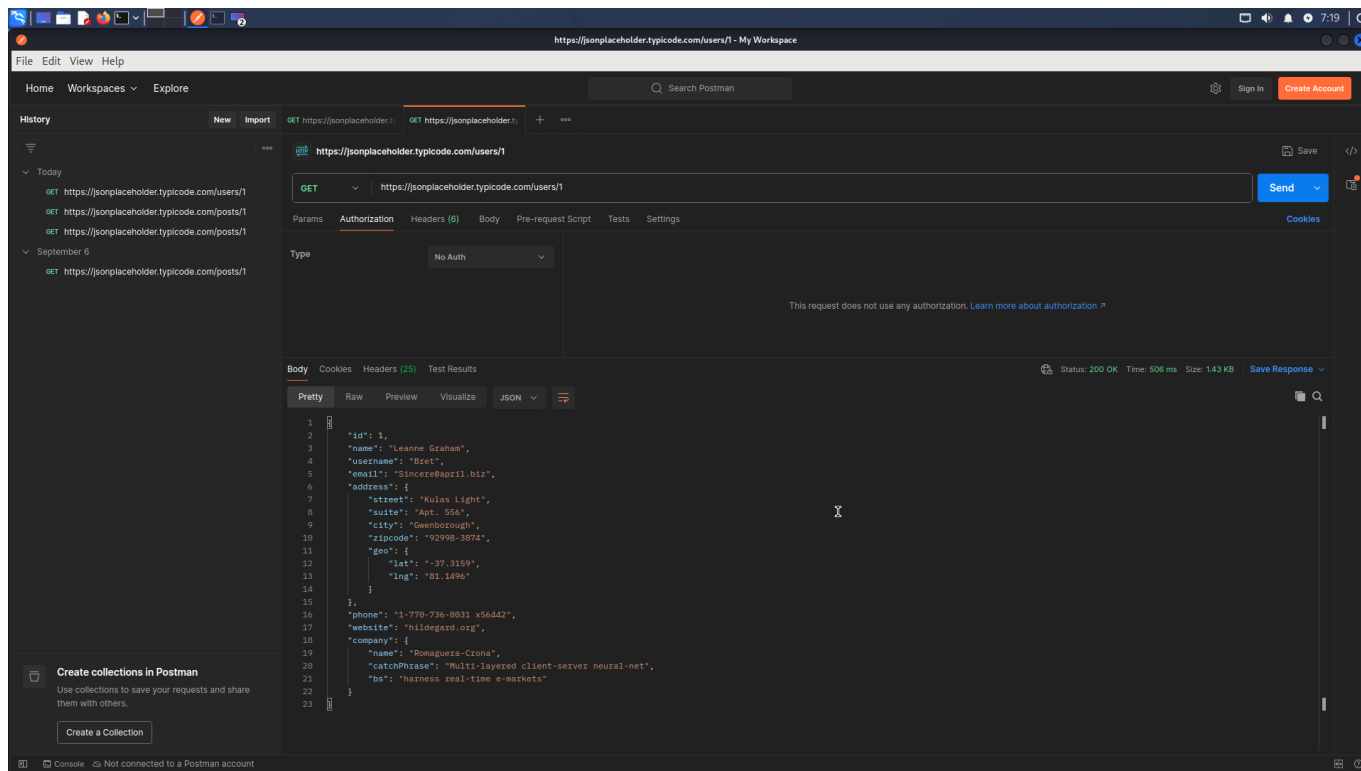


Figure 7. Single user record returned without configured authentication.

Recorded result

GET `https://jsonplaceholder.typicode.com/users/1` returned 200 OK. No Auth is visible. Fields include name, username, email, address, geographic coordinates, phone, website and company.

Security interpretation

The result supports an inventory of potentially sensitive field types, not a finding that real personal data was leaked. For a production service, the business owner would need to identify which fields should be public and which require authorization. This is observation O-01.

E08 User collection response

05_all_users_no_auth_response.png

Compare access to one object with access to a collection of objects.

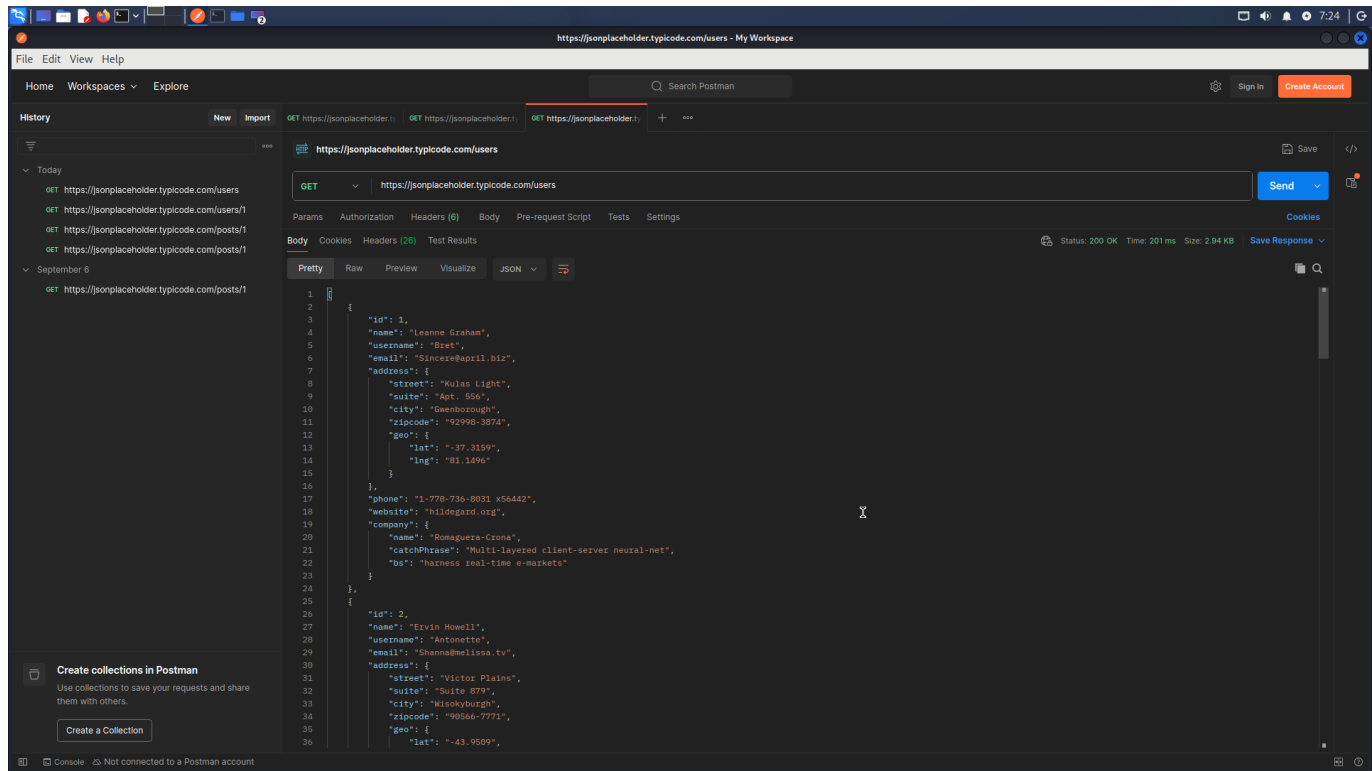


Figure 8. User collection response.

Recorded result

GET `https://jsonplaceholder.typicode.com/users` returned 200 OK and a JSON array. The screenshot shows the first record and the beginning of the second; additional content extends below the visible area.

Security interpretation

The image proves that a collection response was returned, but it is not a full visual record of every element. Collection access can increase exposure in a real system. Here it is documented as public test-data behavior, not bulk theft or a confirmed authorization failure.

E09 Comparison with a second object identifier

06_second_object_access.png

Observe whether changing the public identifier selects a different resource.

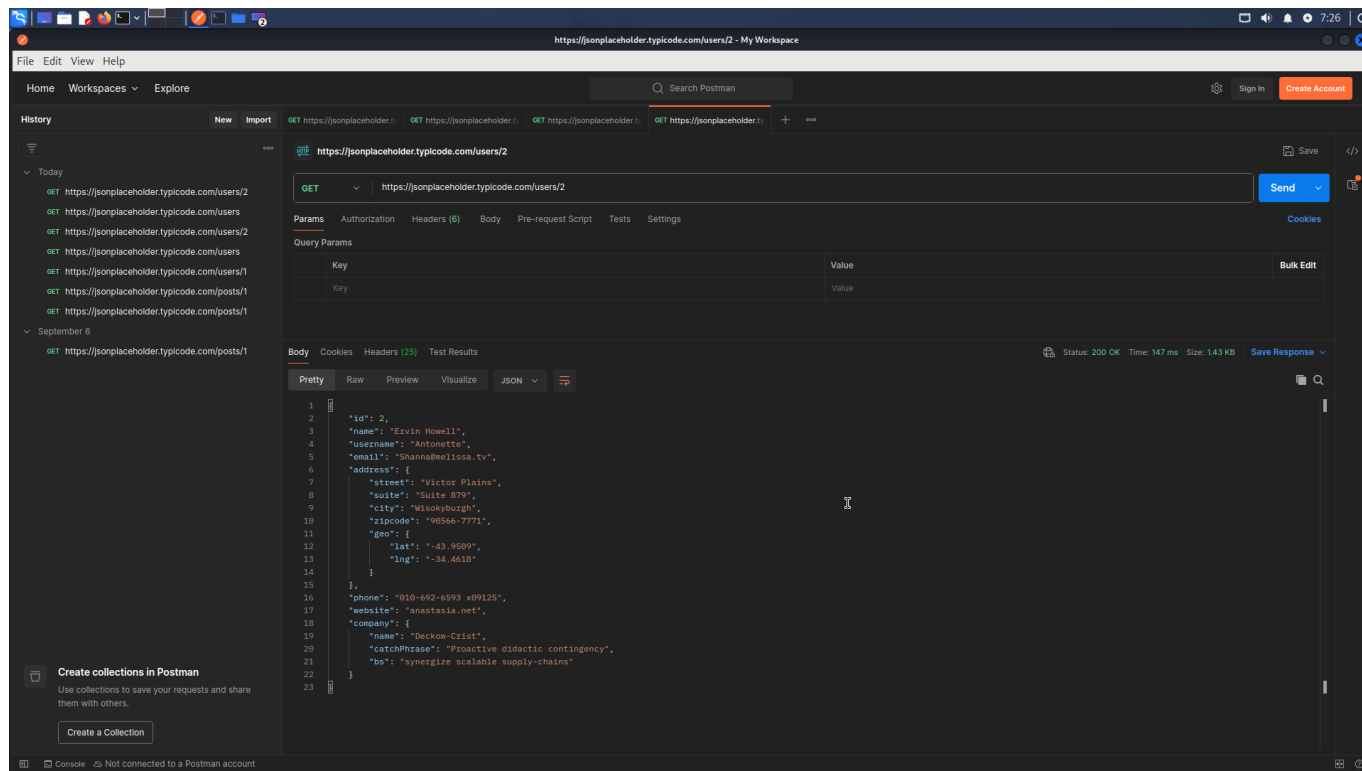


Figure 9. Comparison with a second object identifier.

Recorded result

GET `https://jsonplaceholder.typicode.com/users/2` returned 200 OK and an object with id 2 and name Ervin Howell. This differs from id 1 in E07.

Security interpretation

Changing an identifier is not sufficient to prove Broken Object Level Authorization. That finding requires a rule saying the caller must not access the selected object. No authenticated roles, ownership boundary or tenant boundary was tested in this project.

E10 Nonexistent user identifier

07_nonexistent_user_id.png

Check how the service responds when a requested resource does not exist.

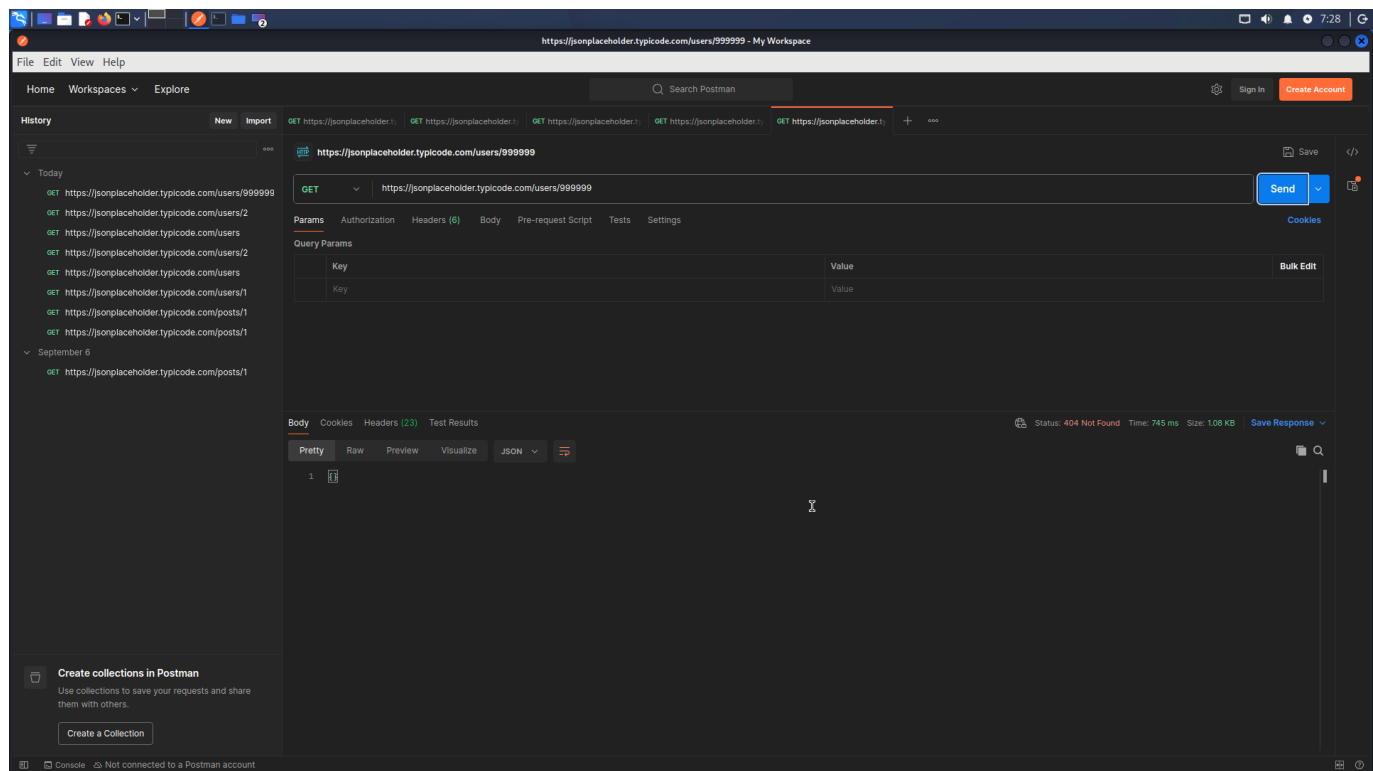


Figure 10. Nonexistent user identifier.

Recorded result

GET `https://jsonplaceholder.typicode.com/users/999999` returned 404 Not Found with an empty JSON object: `{}`.

Security interpretation

The observed failure was controlled and did not expose a stack trace, database query or internal path. This is a positive result for this input only. It does not establish that all invalid inputs or all routes are handled securely.

E11 Nonnumeric user identifier

08_invalid_user_id_format.png

Observe the response to a simple format variation without sending an attack payload.

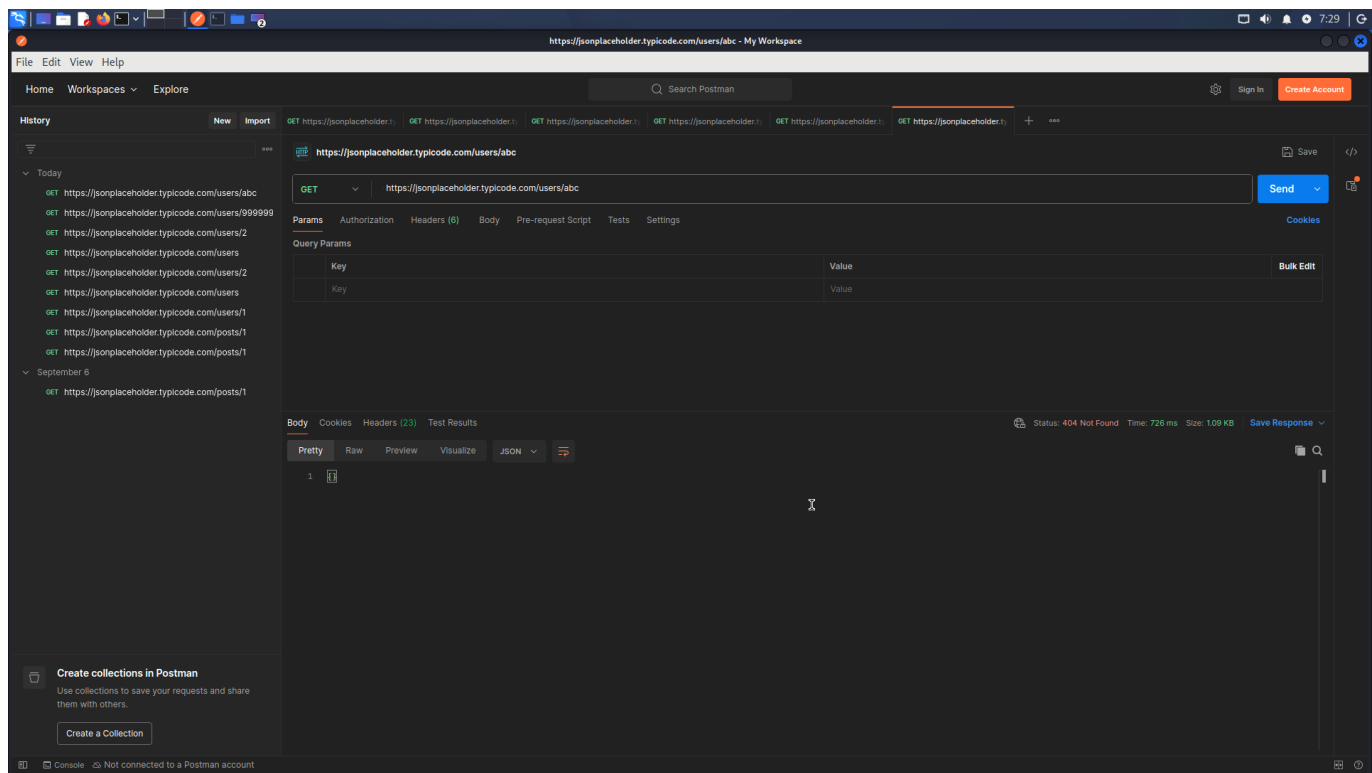


Figure 11. Nonnumeric user identifier.

Recorded result

GET `https://jsonplaceholder.typicode.com/users/abc` returned 404 Not Found with {}.

Security interpretation

The route did not return a user object or a detailed internal error. A 404 can reflect route matching or resource lookup rather than explicit numeric validation. No injection payload, oversized input or automated fuzzing was used, so comprehensive input validation remains unassessed.

E12 Redirect handling for the HTTP check

09a_redirects_disabled.png

Ensure the initial HTTP response is visible instead of automatically following a redirect.

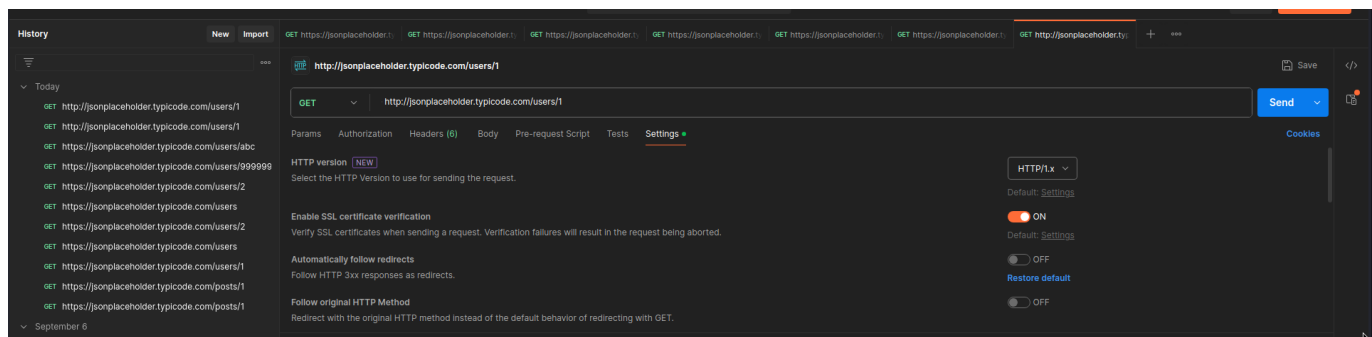


Figure 12. Redirect handling for the HTTP check.

Recorded result

The URL begins with http:// and Automatically follow redirects is OFF. Enable SSL certificate verification is ON in the displayed settings.

Security interpretation

This setting supports the interpretation of E13. If redirects were followed automatically, a final HTTPS 200 could be mistaken for an HTTP 200. Certificate verification is relevant to HTTPS connections; turning it on does not encrypt an HTTP request.

Why the setting matters

The next evidence page contains the response body, followed by both header views. Read these four items together: request setting, response status and body, then response headers.

E13 API body delivered over HTTP

09b_http_200_response_body.png

Determine whether the selected API endpoint serves data directly over unencrypted transport.

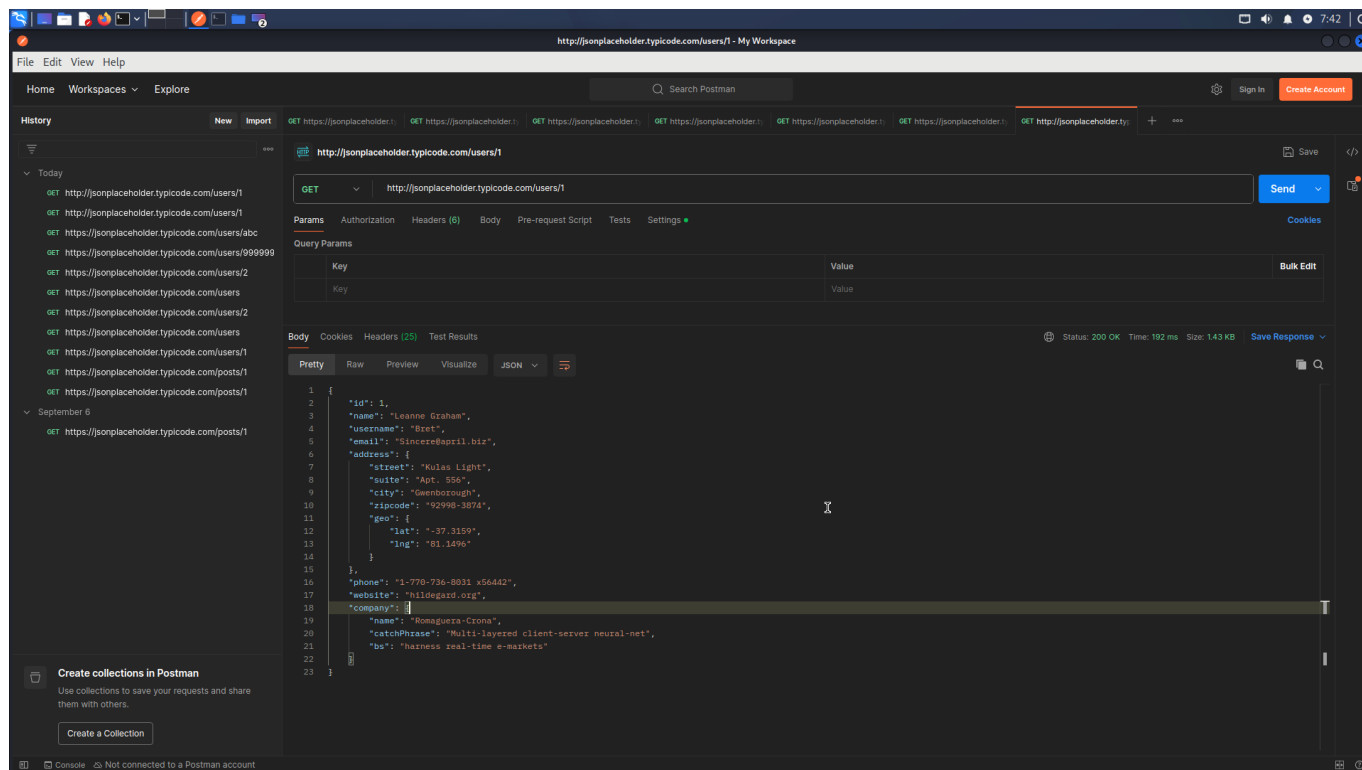


Figure 13. API body delivered over HTTP.

Recorded result

GET `http://jsonplaceholder.typicode.com/users/1` returned 200 OK and the user record. The address shown in Postman remains HTTP.

Security interpretation

Together with E12 and the header views E14-E15, this supports F-01: the tested HTTP endpoint delivered data rather than an HTTPS redirect. No credential was sent and no interception was attempted. The conclusion is restricted to this observed endpoint and response.

E14 HTTP response headers in the upper view

09c_http_response_headers_top.png

Corroborate the HTTP status and inspect the response content type.

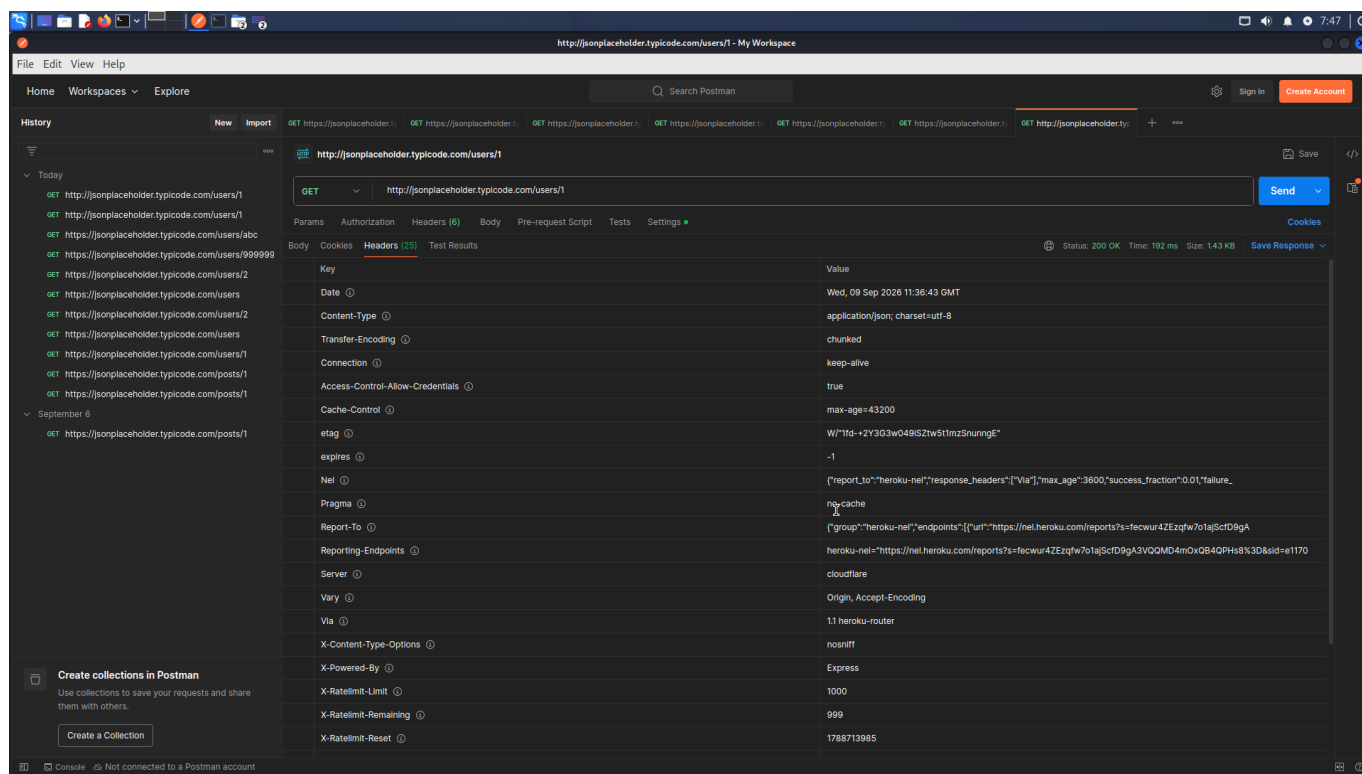


Figure 14. HTTP response headers in the upper view.

Recorded result

The request URL is HTTP, status is 200 OK, and Content-Type is `application/json; charset=utf-8`. The view includes Date, caching metadata, Server, Via, nosniff, Express and rate-limit fields.

Security interpretation

This is response evidence, not proof of a packet capture. It reinforces the transport finding in the client's observed request path. nosniff and a correct content type do not compensate for the lack of transport encryption.

E15 HTTP response headers in the lower view

09d_http_response_headers_bottom.png

Complete the HTTP header evidence and distinguish cached metadata from live controls.

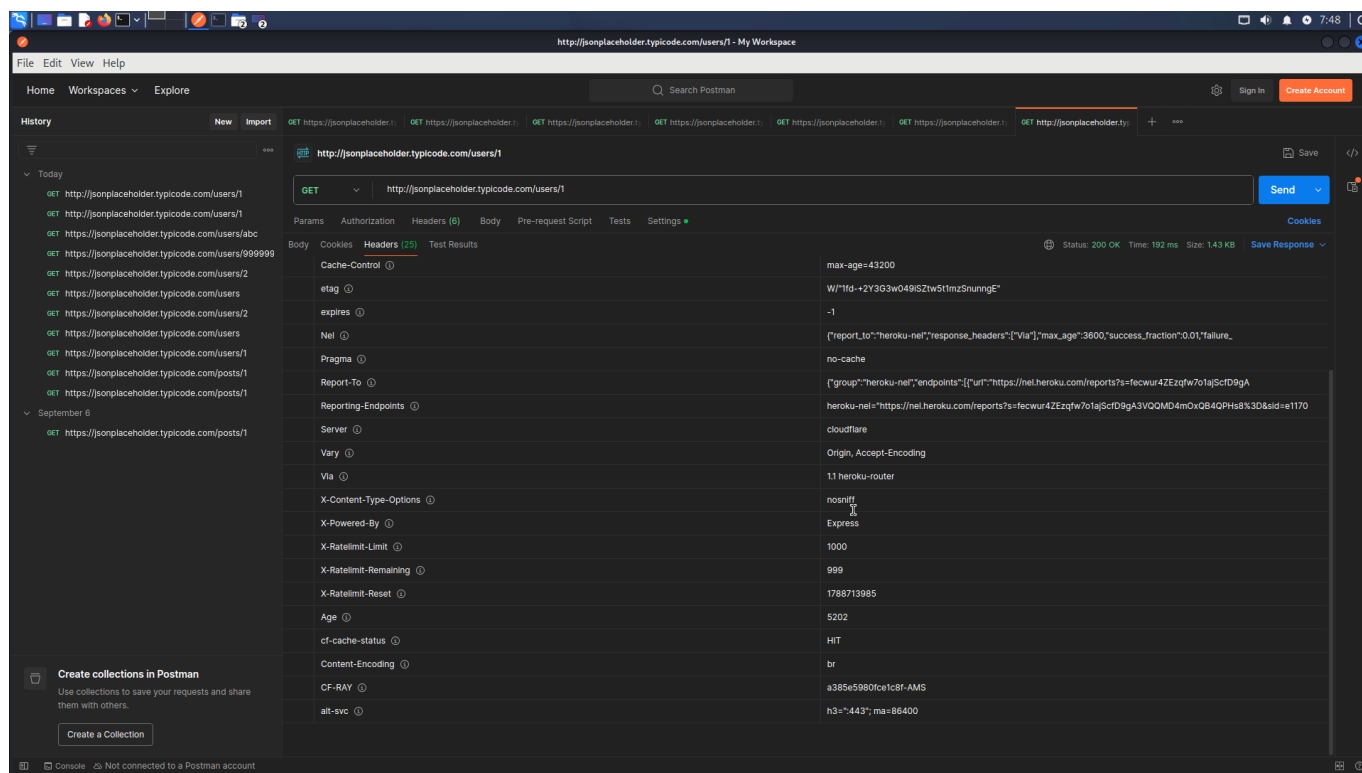


Figure 15. HTTP response headers in the lower view.

Recorded result

The lower view shows Via: 1.1 heroku-router, X-Powered-By: Express, rate-limit values 1000 and 999, Reset: 1788713985, Age: 5202 and cf-cache-status: HIT.

Security interpretation

No HTTPS Location destination is visible across the paired header views; the response status is 200 rather than a redirect. Cache metadata limits what can be inferred about quota counters. The alt-svc advertisement for an alternative connection does not upgrade the HTTP response already received.

E16 ReqRes public endpoint response

10_reqres_no_auth_response.png

Compare a second testing service's public endpoint with JSONPlaceholder.

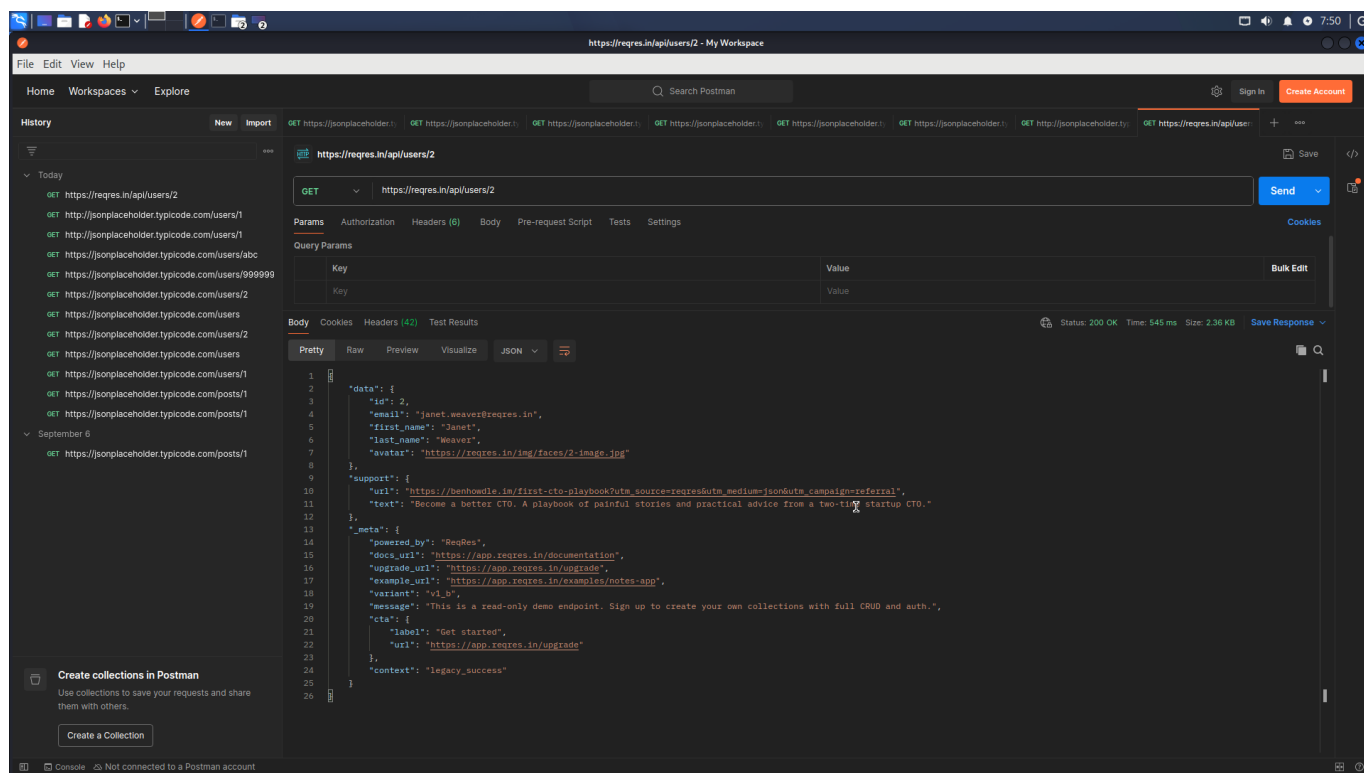


Figure 16. ReqRes public endpoint response.

Recorded result

GET `https://reqres.in/api/users/2` returned 200 OK with a sample user object. The response metadata describes the endpoint as a read-only demo and includes context: `legacy_success`.

Security interpretation

The response itself provides useful design context: this is a public demonstration route. The filename records the test intention, but the displayed Params tab does not independently show the authentication setting. The result must not be reported as an authentication bypass.

E17 ReqRes request and response bodies

10a_reqres_public_endpoint_body.png

Record the distinction between the body sent by the client and the data returned by the server.

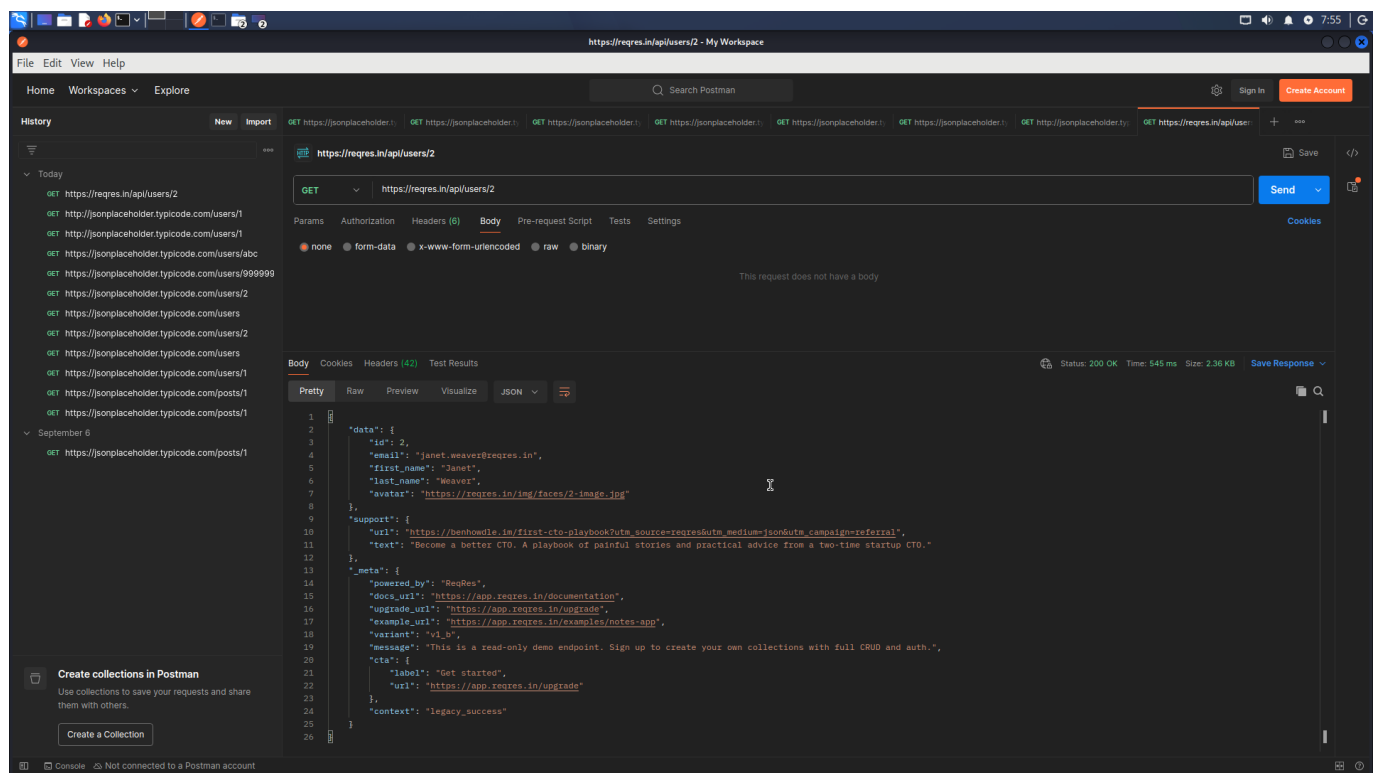


Figure 17. ReqRes request and response bodies.

Recorded result

The request Body tab has none selected. The response contains data with id, email, first_name, last_name and avatar, plus support and service metadata. Status is 200 OK.

Security interpretation

The GET request did not submit a form or JSON payload. Promotional links and documentation references are response content, not authentication tokens. They were inspected as text; no need existed to follow them or authenticate to complete this public response review.

E18 ReqRes response controls and quota metadata

10b_reqres_public_endpoint_headers.png

Inspect the available security and resource-control indicators.

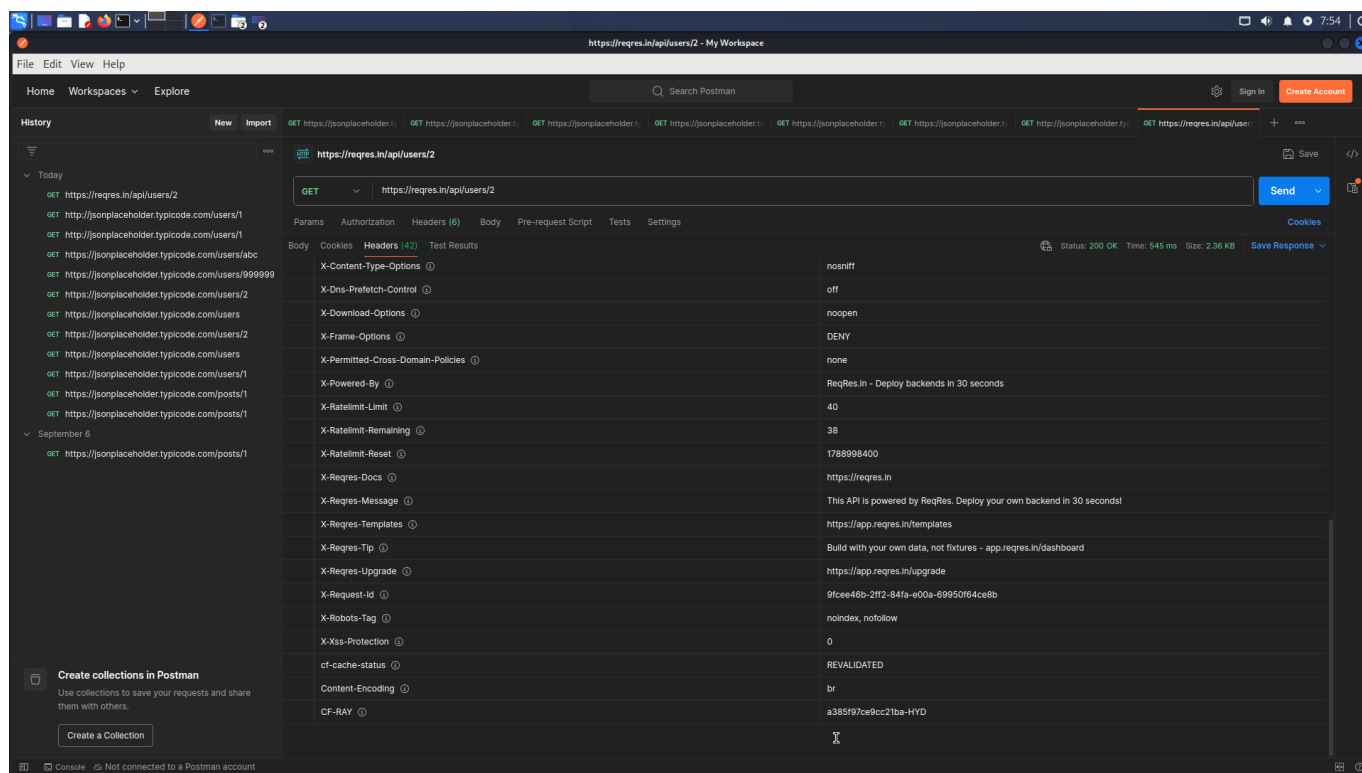


Figure 18. ReqRes response controls and quota metadata.

Recorded result

Visible values include X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-Ratelimit-Limit: 40, Remaining: 38, Reset: 1788998400 and cf-cache-status: REVALIDATED.

Security interpretation

This supports the presence of the listed headers, not an exhaustive audit of the 42-header response. The limit period and effective enforcement were not measured. Browser-oriented headers do not establish API authorization. The X-Powered-By value is a service promotion, not an exposed Express version.

E19 Missing API key rejected by ReqRes

11_reqres_missing_api_key.png

Check normal authentication enforcement without using, guessing or modifying a valid key.

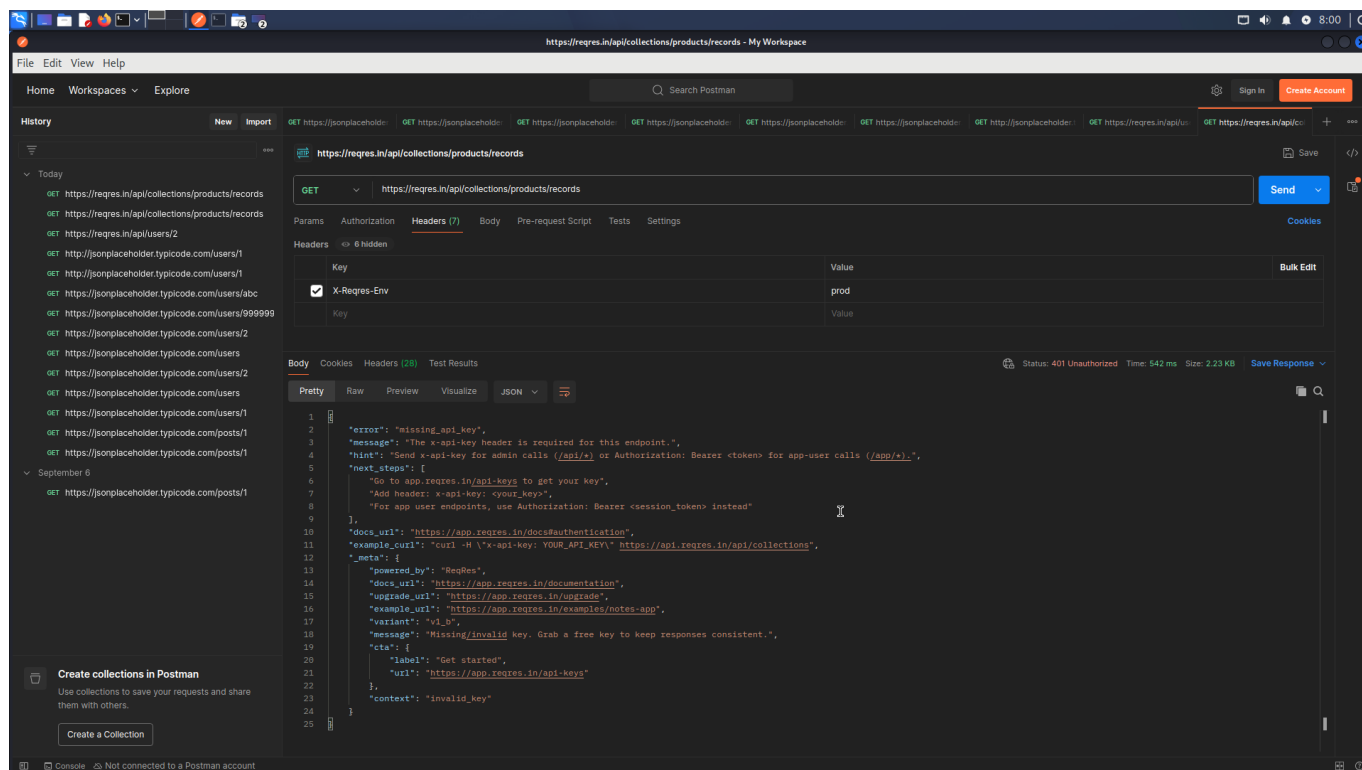


Figure 19. Missing API key rejected by ReqRes.

Recorded result

GET `https://reqres.in/api/collections/products/records` returned 401 Unauthorized. The error is `missing_api_key`. `X-Reqres-Env: prod` is the visible custom request header; the response states that `x-api-key` is required.

Security interpretation

The request was rejected and no collection records were returned. This is a positive authentication observation for one path. The `prod` selector is not a credential, proof of access to a production tenant, or permission for further testing. The result does not prove that the products collection exists or that all authorization controls are correct.